

TASK1

ANALYSIS

System Configuration details

a)Hardware Configuration

OS Name: Microsoft Windows 11 Home Single Language

OS Version: 10.0.26200 N/A Build 26200

CPU Information: 13th Gen Intel(R) Core(TM) i7-13700H with 14 cores and 20 logical processors.

RAM: 16 GB

Storage: HDD, 512 GB

b)Software Configuration

Python Version: 3.12.10

PostgreSQL Version: 17.6

Database: tpch_db lineitem table

c)PostgreSQL Configuration

Number of shared buffers = 4GB

Working memory = 4MB

Maintenance working memory = 64MB

Effective cache size = 4GB

Random page cost = 4

Sequential page cost = 1

Max parallel workers per gather = 2

Planner Settings:

SET enable sequential scan = OFF;

SET enable index scan = ON;

SET enable bitmap scan = ON;

Scenario 1: creating table and adding index before loading data

Scenario 2: creating table loading data and then adding index

Scenario 2 is faster than scenario 1 because of the Maintenance overhead.

In scenario 1 : The B tree variant needed for index has to rebalance itself after every insertion so searching for the key and then rebalancing the tree for every insertion is costly.

In scenario 2 : “Bulk loading” takes place, inserting in bulk opens room for many optimisations, since the table is read at once sequential access is performed where scenario 1 had random accesses.

sorting based on the index can be done for the entire table at once , then inserting the keys from bottom to up ensuring the B tree variant properties avoiding splitting

QUERY SCREEN SHOTS

Scenario 1:

```
tpch_db=# CREATE INDEX I2 ON lineitem2(l_quantity);
CREATE INDEX
Time: 1.331 ms
tpch_db=# \copy lineitem2 FROM 'C:/Users/rmuku/Personal/IITH/Semester 4/CS3563 -
DBMS 2/Assignments/Assignment 1/lineitem_sf10.csv' WITH (FORMAT csv, HEADER);
COPY 59986052
Time: 961667.451 ms (16:01.667)
tpch_db=#
```

Scenario 2:

```
tpch_db=# \copy lineitem1 FROM 'C:/Users/rmuku/Personal/IITH/Semester 4/CS3563 -
DBMS 2/Assignments/Assignment 1/lineitem_sf10.csv' WITH (FORMAT csv, HEADER);
COPY 59986052
Time: 502726.419 ms (08:22.726)
tpch_db=# CREATE INDEX I1 ON lineitem1(l_quantity);
CREATE INDEX
Time: 82508.518 ms (01:22.509)
```

Scenario 1 takes 16 minutes approx

Scenario 2 take 9:45

The justification for Scenario1 being faster than Scenario 2 is written above.